

Especificação de Requisitos de Software

Documento 7 de 8 — Interfaces

Postinder — Plataforma de Gestão e Aprovação de Conteúdo

Agosto de 2026 · Versão 1.0

Sumário

1	Introdução	1
2	Interfaces de usuário	2
2.1	Área Administrativa (apps/admin)	2
2.2	Portal do Cliente	2
3	Interface de programação (API REST)	3
3.1	Autenticação — /api/v1/auth	3
3.2	Identidade Visual — /api/v1/branding	3
3.3	Portal por token — /api/v1/portal/:token	3
3.4	Portal autenticado — /api/v1/client-portal (<i>contexto client</i>)	3
3.5	Postagens — /api/v1/posts (<i>admin</i>)	4
3.6	Clientes — /api/v1/clients (<i>admin</i>)	4
3.7	Usuários — /api/v1/users (<i>admin</i>)	4
3.8	Aprovações, arquivos e feedback — /api/v1/approvals, /api/v1/files, /api/v1/feedback (<i>admin</i>)	4
3.9	Atividades e notificações — /api/v1/activities, /api/v1/notifications (<i>admin</i>)	5
3.10	Integrações — /api/v1/integrations (<i>admin</i>)	5
3.11	Manutenção — /api/v1/maintenance (<i>admin, apenas quando habilitado</i>)	5
3.12	Diagnóstico (<i>públicas</i>)	5
3.13	Formatos de dados e convenções	5
4	Interfaces externas	5
5	Interface de linha de comando (operação)	6

1 Introdução

Este documento descreve as interfaces do Postinder em três camadas: **interfaces de usuário** (área administrativa e portal do Cliente), **interface de programação (API REST)** exposta pelo backend, e **interfaces externas** consumidas pela plataforma. O objetivo é permitir que qualquer equipe de desenvolvimento, QA ou integração compreenda os pontos de contato do sistema sem precisar ler o código-fonte diretamente.

2 Interfaces de usuário

2.1 Área Administrativa (apps/admin)

Aplicação single-page acessada pela equipe da agência, organizada nas seguintes telas principais (src/features):

Tela	Conteúdo
Dashboard	Visão geral operacional, com painéis recolhíveis Atividade recente e Postagens (independentes entre si, expandem sob demanda).
Clientes	Listagem, cadastro, edição, ativação/desativação, exclusão definitiva, gestão do link de portal e da preferência de visualização detalhada.
Postagens	Criação, edição, upload e reordenação de anexos, envio individual/em lote, reenvio após correção, duplicação e registro de execução. Inclui prévia compacta navegável do feed.
Aprovações	Acompanhamento da fila de decisão do Cliente (somente leitura do ponto de vista da decisão, já que a agência não decide em nome do Cliente).
Usuários	Cadastro, edição e remoção de usuários internos e seus papéis (exclusivo de admin).
Insights	Geração de insights administrativos via IA (exclusivo de admin), a partir de métricas agregadas.
Configurações — Identidade Visual (Branding)	Upload, substituição e remoção do logotipo institucional, com prévia e confirmação explícita (exclusivo de admin).
Reset de Ambiente	Disponível apenas quando o ambiente está configurado como demonstração; reinicializa os dados de exemplo.

A área administrativa possui suporte a tema claro e escuro, com paleta de destaque em magenta (navegação, seleção, foco) e cores semânticas reservadas — verde (aprovação), vermelho (recusa/ajuste) e amarelo/laranja (pendência) — que não são substituídas pela cor da marca.

2.2 Portal do Cliente

Disponível em duas rotas com a mesma regra de decisão:

- `/portal/:token` — acesso por link privado (token), sem necessidade de login.
- `/aprovar` — acesso por Cliente autenticado (e-mail e senha).

Modo simplificado (padrão): fila guiada, ordenada por data prevista crescente (itens sem data por último), exibindo uma Postagem pendente por vez; após cada decisão, a fila avança automaticamente até o estado **“Tudo em dia”**.

Modo detalhado (quando `portal_detailed_view = true` para o Cliente): reexibe o seletor de Postagens e o painel **Visão geral / Acompanhamento do conteúdo**, que inicia recolhido e preserva aba ativa, filtros e dados carregados.

Componentes centrais do fluxo de revisão:

- Prévia de mídia compartilhada (imagens e vídeos), com player nativo, controles, `playsInline` e carregamento por metadados para vídeo.
- Gesto de swipe equivalente para imagens e vídeos (esquerda = ajuste, direita = aprovação), com botões explícitos como alternativa acessível.
- Navegação anterior/próximo entre anexos pendentes, sem disparar decisões.
- Legenda alinhada à esquerda, com hifenização em português e expansão “Ver mais”.
- Ações fixas na parte inferior em dispositivos móveis; junto à legenda em telas maiores.
- Painéis complementares (métricas, calendário, histórico, feedbacks) recolhidos por padrão.

3 Interface de programação (API REST)

Todas as rotas administrativas e de portal seguem o prefixo `/api/v1`, respondem em JSON e utilizam os verbos HTTP convencionais. Rotas administrativas exigem cabeçalho de autenticação `Bearer <token>` com contexto `admin` e a capacidade declarada na política de autorização (ver Documento 2). Rotas de portal por token não exigem autenticação de usuário, mas dependem da validade do token na URL; rotas de portal autenticado exigem contexto `client`.

3.1 Autenticação — `/api/v1/auth`

- `POST /login` — autentica usuário administrativo, retorna `access` e `refresh` token.
- `POST /refresh` — renova o `access` token a partir de um `refresh` token válido.
- `POST /logout` — encerra a sessão corrente.

3.2 Identidade Visual — `/api/v1/branding`

- `GET /` (*pública*) — retorna nome institucional, URL utilizável, indicador de configuração, versão e data de atualização.
- `POST /logo` (*admin, capacidade branding:update*) — envia/substitui o logotipo.
- `DELETE /logo` (*admin, capacidade branding:update*) — remove o logotipo configurado.

3.3 Portal por token — `/api/v1/portal/:token`

- `GET /` — dados do portal e do Cliente associado ao token.
- `GET /posts` — lista de Postagens visíveis ao Cliente.
- `POST /posts/:postId/approve` — aprova a Postagem.
- `POST /posts/:postId/reject` — reprova a Postagem.
- `POST /files/:fileId/approve` — aprova um arquivo.
- `POST /files/:fileId/reject` — reprova um arquivo (com motivo/tags).
- `PATCH /files/:fileId/feedback` — edita o feedback de um arquivo já reprovado.
- `POST /files/:fileId/reset` — desfaz a última decisão sobre o arquivo.
- `POST /posts/:postId/soundtrack/approve` — aprova o fundo sonoro da Postagem.
- `POST /posts/:postId/soundtrack/adjust` — solicita ajuste do fundo sonoro (comentário obrigatório).
- `POST /posts/:postId/soundtrack/reset` — desfaz a última decisão sobre o fundo sonoro.
- `POST /feedback` — envia feedback mensal.

Qualquer sub-rota não reconhecida sob `/api/v1/portal/*` responde HTTP 404 genérico (RNF-SEG-07).

3.4 Portal autenticado — `/api/v1/client-portal (contexto client)`

Espelha o mesmo conjunto de operações do portal por token (`GET /`, aprovação/reprovação de Postagens, arquivos e fundo sonoro, edição de feedback, reset da última decisão e envio de

feedback mensal), porém autenticado por JWT de Cliente em vez de token de URL.

3.5 Postagens — `/api/v1/posts (admin)`

- GET / — lista Postagens (filtros por Cliente, status, etc.).
- POST / — cria uma nova Postagem (posts:create).
- POST /send-batch-for-approval — envio em lote (posts:submit).
- GET /:id — detalhe da Postagem (posts:read).
- PUT /:id — edita a Postagem (posts:update).
- DELETE /:id — exclui logicamente a Postagem (posts:delete).
- POST /:id/duplicate — duplica a Postagem (posts:duplicate).
- GET /:id/soundtrack / PUT /:id/soundtrack — consulta/edita o fundo sonoro (soundtracks:read / soundtracks:update).
- POST /:id/soundtrack/file — envia o arquivo de áudio do fundo sonoro (soundtracks:upload).
- PATCH /:id/status — alterna exclusivamente entre draft e ready (posts:change-status).
- POST /:id/execute — registra a execução/publicação (posts:execute).
- POST /:id/files — envia anexos (files:upload).
- PATCH /:id/files/reorder — reordena anexos (files:reorder).
- POST /:id/files/:fileId/replace — substitui um anexo (files:replace).
- DELETE /:id/files/:fileId — remove um anexo (files:delete).
- POST /:id/submit-for-approval e POST /:id/send-for-approval — envia para aprovação (posts:submit).
- POST /:id/resubmit — reenvia após correção (posts:resubmit).

3.6 Clientes — `/api/v1/clients (admin)`

- GET /, POST /, GET /:id, PUT /:id — CRUD básico (clients:read / clients:create / clients:update).
- POST /:id/notify — notifica o Cliente (clients:notify).
- PATCH /:id/activate — reativa o Cliente (clients:update).
- GET /:id/portal-link, POST /:id/portal-link, POST /:id/portal-link/replace — consulta, geração e substituição do link do portal (clients:portal-access).
- DELETE /:id — desativação (clients:deactivate).
- DELETE /:id/permanent — exclusão definitiva (clients:delete).

3.7 Usuários — `/api/v1/users (admin)`

- GET /, POST /, PUT /:id, DELETE /:id — CRUD de usuários internos (admin-users:read / create / update / delete, todas exclusivas de admin).

3.8 Aprovações, arquivos e feedback — `/api/v1/approvals, /api/v1/files, /api/v1/feedback (admin)`

- GET /approvals/queue — fila administrativa de acompanhamento (approvals:read).
- POST /files/:id/approve, POST /files/:id/reject — ações administrativas equivalentes de suporte (files:review).
- GET /feedback/monthly — métricas de feedback mensal (metrics:read).
- POST /feedback — registro administrativo de feedback (feedback:create).

3.9 Atividades e notificações — /api/v1/activities, /api/v1/notifications (*admin*)

- GET /activities, POST /activities — consulta e registro de eventos de auditoria (activities:read / activities:create).
- GET /notifications — lista notificações (notifications:read).
- POST /notifications/read, POST /notifications/read-all — marca como lida (notifications:update).

3.10 Integrações — /api/v1/integrations (*admin*)

- POST /ai-insights — gera um insight administrativo via IA (ai-insights:generate).

3.11 Manutenção — /api/v1/maintenance (*admin, apenas quando habilitado*)

- POST /reset-demo-data — restaura os dados de demonstração (demo-reset:execute); rota inexistente (HTTP 404) quando o ambiente não está configurado como demonstração.

3.12 Diagnóstico (*públicas*)

- GET /health — disponibilidade geral do processo.
- GET /health/db — conectividade com PostgreSQL.
- GET /health/storage — disponibilidade do Storage (local em desenvolvimento; Supabase em produção).

3.13 Formatos de dados e convenções

- Corpo de requisição e resposta em **JSON** (Content-Type: application/json), exceto uploads de arquivo, que utilizam **multipart/form-data**.
- Erros seguem o formato { "error": "<mensagem>", "code": "<código>" } quando aplicável (ex.: FILE_TOO_LARGE, UNSUPPORTED_FILE_TYPE, NOT_FOUND).
- Códigos HTTP relevantes: 200/201 sucesso; 403 autorização negada (capacidade ausente); 404 recurso ou rota inexistente; 413 arquivo excede o limite de tamanho; 415 tipo de arquivo não suportado; 503 indisponibilidade segura (ex.: IA sem credencial configurada, validação de branding ocupada).

4 Interfaces externas

Sistema externo	Protocolo	Sentido	Descrição
Anthropic API	HTTPS, server-side	Postinder → Anthropic	Geração de insights administrativos a partir de dados agregados e pseudonimizados; modelo restrito por allowlist, com timeout e cancelamento.

Sistema externo	Protocolo	Sentido	Descrição
Supabase Storage	HTTPS (Storage API)	Postinder ↔ Supabase	Armazenamento de arquivos de mídia, fundo sonoro e logotipo institucional em produção.
Supabase PostgreSQL	TCP/TLS (protocolo pg)	Postinder ↔ Supabase	Persistência relacional principal em produção.
Z-API (WhatsApp)	HTTPS, server-side (planejada)	Postinder → Z-API	Estrutura prevista para notificações via WhatsApp; ainda não habilitada nas rotas atuais.
Google Analytics	Client-side (script público)	Frontend → Google	Apenas identificador público de medição (VITE_GA_MEASUREMENT_ID); nenhum dado sensível é enviado por esta via.
Twilio / GoHighLevel / Canva / Resend	HTTPS, server-side (planejadas)	Postinder → provedor	Integrações previstas na arquitetura, porém desabilitadas até a implementação de fluxos seguros (OAuth/server-side).

5 Interface de linha de comando (operação)

Além das interfaces de usuário e de API, o backend expõe comandos de operação executados via `npm run <comando>` a partir do diretório backend, relevantes para equipes de infraestrutura e operação:

Comando	Finalidade
<code>db:migrate</code>	Aplica migrations pendentes (única fonte de verdade estrutural).
<code>db:seed-demo</code>	Popula dados de demonstração (requer <code>APP_MODE=demo</code>).
<code>db:bootstrap-admin</code>	Cria o primeiro administrador a partir de variáveis de ambiente.
<code>storage:cleanup-retention</code>	Remove objetos de Storage vencidos conforme a política de retenção das Postagens.
<code>db:validate-email-uniqueness,</code> <code>db:validate-storage-metadata,</code> <code>db:validate-storage-duplication,</code> <code>db:audit-shared-storage-files,</code> <code>db:validate-storage-retention,</code> <code>db:cleanup-legacy-archived</code>	Scripts de validação e auditoria técnica do estado do banco/Storage.